Simulering av kollisjoner mellom partikler

Olai Solsvik

Innhold

1 Simulering i én dimensjon 1
1.1 Teori 1
1.2 Implementasjon 1
1.3 Utregning av minsteavstand 2
1.4 Bevaring av energi 3
1.5 Kollisjoner mellom andre nuklider 4
2 Simulering i 2D 5
2.1 Teori 5
2.2 Implementasjon 5
2.3 Utregning av vinkel 6
2.4 Generalisering av funksjonen 6
2.5 Simulering med flere ulike startverdier 7
3 Konklusjon 7

1 Simulering i én dimensjon

1.1 Teori

I denne oppgaven skal jeg ved hjelp av Coulombs lov simulere kollisjonen mellom to partikler. Jeg starter ved å sette opp uttrykket for Coulombs lov:

$$F = k\frac{|q_1 \cdot q_2|}{r^2}$$

Merk

k er Coulombs konstant, definert som

$$k = 8.99 \cdot 10^9\,\mathrm{N} \cdot \mathrm{m}^2 \cdot \mathrm{C}^{-2}$$

Videre kombinerer jeg dette med Newtons andre lov:

$$F = m \cdot a$$

Og finner et uttrykk for akselerasjonen

$$m \cdot a = k\frac{|q_1 \cdot q_2|}{r^2}$$

$$a = k\frac{|q_1 \cdot q_2|}{r^2 \cdot m}$$

Jeg bruker det at $a = r''(t)$ og setter det opp som en differensiallikning:

$$r'' = k\frac{|q_1 \cdot q_2|}{r^2 \cdot m}$$

1.2 Implementasjon

I dette tilfellet velger jeg å simulere to protoner som går mot hverandre. Jeg starter dem parallelt om sentrum med posisjon $2 \cdot 10^{-10}$ i hver retning. Startfarten til partiklene er satt som $5 \cdot 10^4\,\mathrm{m} \cdot \mathrm{s}^{-1}$. Kollisjonen skjer innen et ekstremt lite tidsrom.

Dette kan jeg implementere med programmering ved hjelp av eulers metode som bruker numerisk integrasjon som følger. Her bruker jeg også dynamisk programering (DP) som en metode for å effektivisere den kjøringen som i utgangspunktet er definert som en rekursiv funksjon. Med dynamisk programmering kan man løse dette i lineær tid.

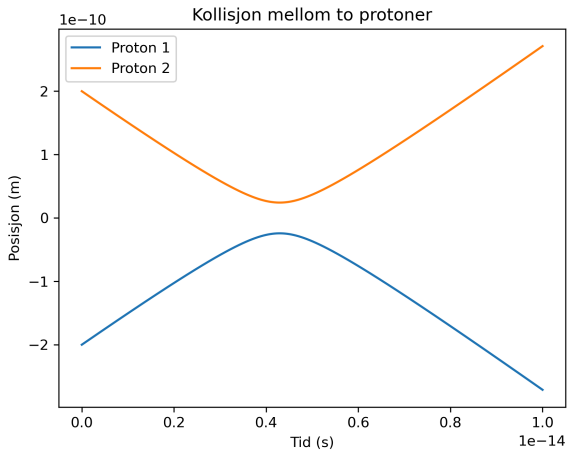
```python
# Importere biblioteker og konstanter
from matplotlib import pyplot as plt
from scipy import constants
import numpy as np

k = 8.99e9

m1, q1 = constants.m_p, constants.e # Proton
m2, q2 = constants.m_p, constants.e # Proton

# Definere tidsintervallet.
# Dette skjer over et ekstremt lite tidsrom.
dt = 1e-18
T = 1e-14
MAXN = int(T / dt) + 1

# Definere DP-tabeller
ts = np.zeros(MAXN)
delta_r = np.zeros(MAXN)
r1 = np.zeros(MAXN)
r2 = np.zeros(MAXN)
dr1 = np.zeros(MAXN)
dr2 = np.zeros(MAXN)
ddr1 = np.zeros(MAXN)
ddr2 = np.zeros(MAXN)

# Startverdiene (forklart over)
r1[0] = -2e-10 # Begynner på venstre side
r2[0] = 2e-10 # Begynner på høyre side
dr1[0] = 5e4 # Bevegelsesretning mot høyre
dr2[0] = -5e4 # Bevegelsesretning mot venstre

# Beregne de andre startverdiene basert på dette
delta_r[0] = abs(r2[0] - r1[0])

# Fylle inn DP-tabellene
n = 0
while n < MAXN - 1:
    n += 1
    ts[n] = ts[n-1] + dt

    delta_r[n] = abs(r2[n-1] - r1[n-1])
    F = k * (q1 * q2) / delta_r[n] ** 2

    # Setter motsatt fortegn på kreftene
    ddr1[n] = -F / m1
    ddr2[n] = F / m2

    dr1[n] = dr1[n-1] + ddr1[n] * dt
    dr2[n] = dr2[n-1] + ddr2[n] * dt

    r1[n] = r1[n-1] + dr1[n] * dt
    r2[n] = r2[n-1] + dr2[n] * dt
```

Merk

Teknisk sett er dette Euler-Cromers («semi-implisitt») metode. Forskjellen er at Eulers metode ville regnet posisjonen basert på forrige tidssteg: $r_n = r_{n-1} + r'_{n-1}\Delta t$, mens Euler-Cromers metode bruker resultatet fra nåværende iterasjon i stedet: $r_n = r_{n-1} + r'_n\Delta t$. Sistnevnte gjør den mer eksakt for slike applikasjoner som dette, og gjør at energi bevares korrekt over tid i systemet.

Jeg kan nå plotte resultatene:

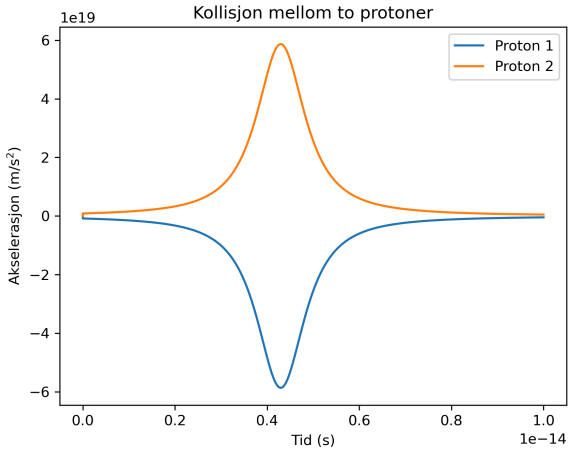
```
fig, ax = plt.subplots()
ax.plot(ts, r1, label="Proton 1")
ax.plot(ts, r2, label="Proton 2")
ax.set_title("Kollisjon mellom to protoner")
ax.set_xlabel("Tid (s)")
ax.set_ylabel("Posisjon 1(m)")
ax.legend()
plt.show()
```



Figur 1: Visualisering av kollisjonen mellom to protoner

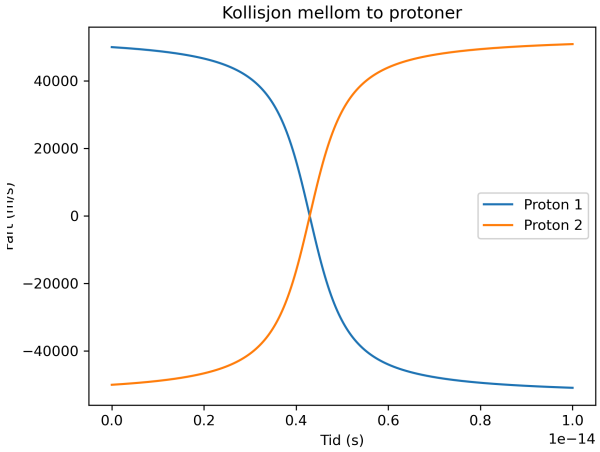
Her ser jeg at de to protonene går mot hverandre nærmest lineært. Mens de nærmer seg hverandre, reduseres farten gradvis, helt til punktet der den snur og akselererer før den beveger seg lineært. Dette skjer når den kommer ut av rekkevidden til de elektriske kreftene som virker mellom partiklene. Da er det ikke lenger noen krefter som virker mellom dem, og da vil partikkelen etter Newtons første lov fortsette å bevege seg med konstant hastighet. Den store krummingen komme fra den sterke repulsive kraften i Coulombs lov, som er proporsjonal med r^{-2}.

Dette kan vi også se tydelig når vi plotter akselerasjonen:



Figur 2: Akselerasjon i kollisjonen

Her har akselerasjonen en horisontal asymptote på $y = 0$. Det vil si at partiklene kun akselerer mens de er innenfor rekkevidden av det elektriske feltet. Siden massen til partiklene i dette tilfellet er lik, og bevegelsesmengden er bevart, vil farten være lik (men negativ) etter kollisjonen:



Figur 3: Fart i kollisjonen

1.3 Utregning av minsteavstand

Jeg vil nå beregne den korteste avstanden r mellom partiklene før de snur retning. Da bruker jeg loven for bevaring av energi $\sum E_{\text{start}} = \sum E_{\text{slutt}}$, sammen med Coulombs lov for potensiell energi:

$$E_P = k \cdot \frac{q_1 \cdot q_2}{r}$$

Merk

Her tar jeg ikke absoluttverdien, altså avhenger den potensielle energien av fortegnet til ladningene.

Jeg beregner da først energien før støtet:

$$E_{\text{start}} = \frac{1}{2}m_1v_1^2 + \frac{1}{2}m_2v_2^2 + k \cdot \frac{q_1 \cdot q_2}{r_{\text{start}}}$$

Deretter skal jeg finne ut hvor mye energi det er etter støtet. Hvis partiklene har lik masse og ladning vil jeg kunne se bort fra den kinetiske energien, da denne er lik 0 i vendepunktet. Hvis de derimot er ulike, må jeg også beregne dette med. Dette kan betraktes som et fullstendig uelastisk støt i øyeblikket støtet skjer, og de to partiklene vil få felles masse og fart. Da kan jeg bruke bevaringsloven for bevegelsesmengde til å finne den felles farten til partiklene:

$$\begin{aligned} p_{\text{start}} &= m_1 \cdot v_{1_{\text{start}}} + m_2 \cdot v_{2_{\text{start}}} \\ p_{\text{minsteavstand}} &= (m_1 + m_2) \cdot v_{\text{felles}} \\ p_{\text{start}} &= p_{\text{minsteavstand}} \\ m_1 \cdot v_{1_{\text{start}}} + m_2 \cdot v_{2_{\text{start}}} &= (m_1 + m_2) \cdot v_{\text{felles}} \\ v_{\text{felles}} &= \frac{m_1 \cdot v_{1_{\text{start}}} + m_2 \cdot v_{2_{\text{start}}}}{m_1 + m_2} \end{aligned}$$

Dette kan jeg bruke sammen med likningen for kinetisk energi for å sette opp den totale energien ved minsteavstand.

$$\begin{aligned} E_{K_{\text{slutt}}} &= \frac{1}{2}(m_1 + m_2) \cdot v_{\text{felles}}^2 \\ E_{\text{slutt}} &= E_{K_{\text{slutt}}} + k \cdot \frac{q_1 \cdot q_2}{r_{\text{min}}} \end{aligned}$$

Deretter setter jeg den totale energien ved start lik den totale energien ved minsteavstand og løser for r_{min}

$$\begin{aligned} E_{\text{start}} &= E_{\text{slutt}} \\ E_{\text{start}} &= E_{K_{\text{slutt}}} + k \cdot \frac{q_1 \cdot q_2}{r_{\text{min}}} \\ E_{\text{start}} - E_{K_{\text{slutt}}} &= k \cdot \frac{q_1 \cdot q_2}{r_{\text{min}}} \\ \left(E_{\text{start}} - E_{K_{\text{slutt}}}\right) \cdot r_{\text{min}} &= k \cdot q_1 \cdot q_2 \\ r_{\text{min}} &= \frac{k \cdot q_1 \cdot q_2}{E_{\text{start}} - E_{K_{\text{slutt}}}} \end{aligned}$$

Dette kan jeg da legge til i koden for å finne minsteavstanden.

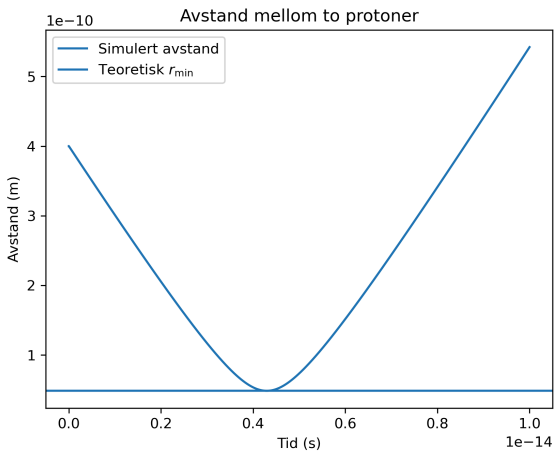
```
Estart = 1/2 * m1 * dr1[0] ** 2 + 1/2 * m2 * dr2[0] ** 2 +
k * (q1 * q2)/(abs(r2[0] - r1[0]))
vfelles = (m1 * dr1[0] + m2 * dr2[0]) / (m1 + m2)
EKslutt = 1/2 * (m1 + m2) * vfelles ** 2
rmin = (k * q1 * q2) / (Estart - EKslutt)
```

Denne koden gir resultatet

$$r_{\text{min}} = 4.849669142782682 \cdot 10^{-11}\ \text{m}$$

Jeg plotter så med følgende kode

```
fig, ax = plt.subplots()
ax.plot(ts, delta_r, label="Simulert avstand")
ax.axhline(y=rmin, label=r"Teoretisk $r_{\text{min}}$")
ax.set_title("Avstand mellom to protoner")
ax.set_xlabel("Tid (s)")
ax.set_ylabel("Avstand (m)")
ax.legend()
plt.show()
```



Figur 4: Minsteavstanden mellom protonene

Jeg kan da bekrefte fra den horisontale linjen at avstanden som beregnes fra simuleringen min samsvarer med det jeg kommer frem til via den teoretiske utregningen.

1.4 Bevaring av energi

Til slutt vil jeg vise at energien er bevart i kollisjonen. For å gjøre dette, igjen likningen for kinetisk energi kombinert med Coulombs lov for potensiell elektrisk energi:

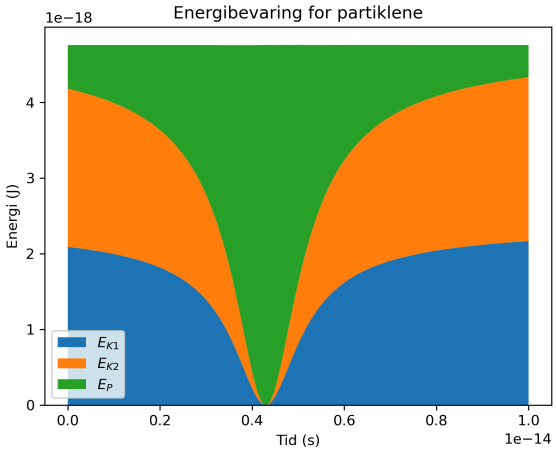
$$\begin{aligned} \sum E_K &= \frac{1}{2} \cdot m_1 \cdot v_1^2 + \frac{1}{2} \cdot m_2 \cdot v_2^2 \\ &= \frac{1}{2} \cdot m_1 \cdot r_1'^2 + \frac{1}{2} \cdot m_2 \cdot r_2'^2 \\ E_P &= k \cdot \frac{q_1 \cdot q_2}{r} \\ E_K + E_P &= \text{konstant} \end{aligned}$$

Jeg legger da til tre flere DP-tabeller i koden for å beregne den kinetiske energien til de to partiklene samt den potensielle elektriske energien mellom dem.

```
# Legge til DP-tabeller
ek1 = np.zeros(MAXN)
ek2 = np.zeros(MAXN)
ep = np.zeros(MAXN)

# Regne ut startbetingelser
ek1[0] = 1/2 * m1 * dr1[0] ** 2
ek2[0] = 1/2 * m2 * dr2[0] ** 2
ep[0] = k * (q1 * q2) / delta_r[0]

while n < MAXN - 1:
    # ... eksisterende kode
```

```
# Beregne kinetisk og potensiell energi
ek1[n] = 1/2 * m1 * dr1[n] ** 2
ek2[n] = 1/2 * m2 * dr2[n] ** 2
ep[n] = k * (q1 * q2) / (delta_r[n])
```

Videre plotter jeg summen av disse verdiene gjennom simuleringen:



Figur 5: Energibevaring i kollisjonen

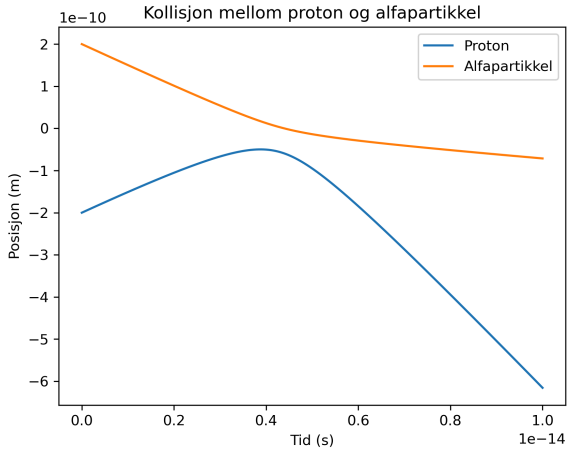
Denne figuren viser den kinetiske energien til det første protonet, det andre protonet, og den potensielle energien mellom de to protonene i henholdsvis blå, oransje og grønn farge. Vi kan se at i kollisjonsøyeblikket, der farten er lik 0, er den kinetiske energien lik 0, mens den potensielle energien er høy. Deretter ser vi at den potensielle energien blir gjort tilbake om til kinetisk energi idet partiklene beveger ser fra hverandre igjen. Viktigst av alt kan vi her se hvordan summen av energien holder seg konstant gjennom hele simuleringen. Dette bekrefter teorien om at energisummen er bevart.

1.5 Kollisjoner mellom andre nuklider

Videre kan jeg bruke samme koden til å simulere andre kollisjoner, for eksempel kollisjonen mellom et proton og en alfapartikkel. Jeg bruker da den samme koden som over, men endrer verdiene for massen og ladningen til partiklene. I dette tilfellet vil jeg se på kollisjonen mellom et proton og en alfapartikkel.

```
m1, q1 = constants.m_p, constants.e
m2, q2 = 2 * constants.m_p + 2 * constants.m_n, 2 *
constants.e
```

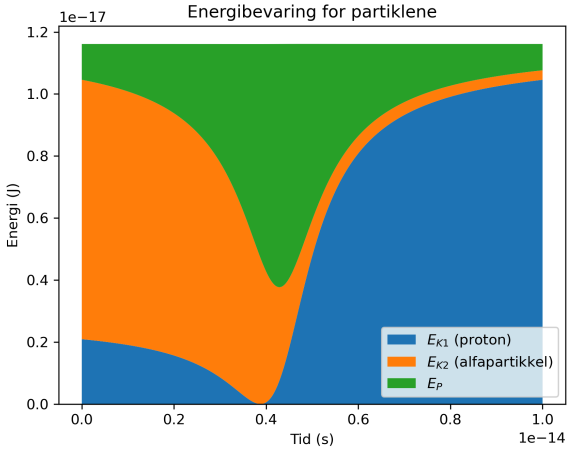
Dette gir mer interessante resultater. Posisjonen til partiklene over tid ser nå slik ut:



Figur 6: Posisjonen til partiklene over tid

Her kan vi se at alfapartikkelen med omtrent $4\times$ så stor masse, får en mye lavere fart etter kollisjonen enn protonet. Dette skjer på grunn av bevaring av bevegelsesmengden.

Energien er fortsatt bevart, men vi ser også her en mer interessant fordeling.



Figur 7: Energibevaring i kollisjone med alfapartikkel

Siden partiklene starter med lik fart, vil alfapartikkelen med $4\times$ så stor masse også ha $4\times$ så stor kinetisk energi. Etter støtet overføres mye av bevegelsesmengden fra alfapartikkelen til det mye letter protonet. For at bevegelsesmengden skal bli bevart må da protonet skytes ut med en mye høyere fart. Siden den kinetiske energien øker kvadratisk med farten ($E = \frac{1}{2}mv^2$) har protonet mot slutten mye mer kinetisk energi enn alfapartikkelen.

Merk

Dette gjelder spesifikt for en sentral kollisjon, som er det jeg simulerer her når jeg kun regner med én dimensjon. Om alfapartikkelen ikke hadde truffet i sentrum av protonet, ville den beholdt mye mer av energien enn når de kolliderer rett på hverandre.

2 Simulering i 2D

Basert på den samme logikken, vil jeg nå utvide programmet mitt til å simulere Rutherfords forsøk. Forsøket går ut på at alfapartikler sendes mot en gullkjerne, og så ser man på hvordan alfapartikkelen avbøyes. Dette beviste da at et atom ikke er en «rosinbolle», slik man trodde før. For å gjøre dette, må jeg utvide implementasjonen min til å fungere i to dimensjoner. Da beregner jeg nå avstanden ved hjelp av pytagoras med $r = \sqrt{\Delta x^2 + \Delta y^2}$, og kraften F må dekomponeres for å kunne finne akselerasjonen (og dermed også fart og posisjon ved hjelp av numerisk integrasjon). Så kan jeg plotte banen som en kurve med x- og y- posisjonene til partiklene. Ved å variere avstanden fra sentrumlinjen som alfapartikkelen sendes inn, kan jeg plotte de ulike banene den vil avbøyes langs, slik som i Rutherfords forsøk.

2.1 Teori

Jeg bruker fortsatt Coulombs lov til dette, definert ved

$$|F| = k\frac{|q_1 \cdot q_2|}{r^2}$$

Forskjellen er at når jeg skal simulere i to dimensjoner, kan jeg ikke lenger bruke kraften som en skalar. Nå må jeg skrive dette som en vektor. Jeg kan da skrive

$$\overrightarrow{\Delta r} = r_2 - r_1 = [x_2 - x_1, y_2 - y_1]$$
$$r = |\overrightarrow{\Delta r}| = \sqrt{\Delta x^2 + \Delta y^2}$$

Deretter finner jeg en enhetsvektor (altså en vektor med lengde 1), og multipliserer dette med magnituden som jeg fikk fra Coulombs lov. Dette gir meg

$$\hat{r} = \frac{\overrightarrow{\Delta r}}{r}$$
$$F = |F| \cdot \hat{r}$$

2.2 Implementasjon

Med dette kan jeg bruke den samme Ruler-Cromer logikken for den samme differensiallikningen som tidligere, der den eneste forskjellen er at jeg nå bruker vektorer i stedet for skalarer. Da vil også tabellene mine inneholde todimensjonale punkter i stedet for en skalar.

I Rutherfords forsøk ble en alfapartikkel ($^4_2\mathrm{He}^{2+}$) sendt mot en gullkjerne ($^{197}_{79}\mathrm{Au}$). Selv om det i forsøket ble brukt et nøytralt gullatom, simulerer jeg her i stedet $^{197}_{79}\mathrm{Au}^{79+}$. Dette er fordi at med farten alfapartikkelen bevegde seg med i det originale eksperimentet ($1.5 \cdot 10^7\,\mathrm{m \cdot s^{-1}}$) var den kinetiske energien svært stor ($\approx 5\,\mathrm{MeV}$), som er ekstremt mye sterkere enn bindingsenergien til elektronene, der selv de innerste kun er bundet med omtrent $80\,\mathrm{keV}$. I tillegg er Bohr-radien omtrent $5.29 \cdot 10^{-11}\,\mathrm{m}$. Min simulering starter alfapartikkelen med en avstand fra kjernen $r = 4 \cdot 10^{-11}\,\mathrm{m}$. Dette er allerede innenfor elektronene, så det vil derfor uansett ikke ha noe å si. Altså ville forskjellen vært negligerbar hvis jeg hadde inkludert elektronene, som er det samme resultatet Rutherford fikk basert på sine eksperimenter.

Merk

I denne implementasjonen simulerer jeg begge partiklene. Da gullkjernen har mye større masse enn alfapartikkelen, ville det vært mulig å forenkle ved å anta at denne står fast i origo, da den store masseforskjellen vil gjøre at den dyttes minimalt i forhold til alfapartikkelen. Likevel har jeg valgt å beholde denne kompleksiteten for å få mer nøyaktige resultater selv om det i praksis er negligerbart.

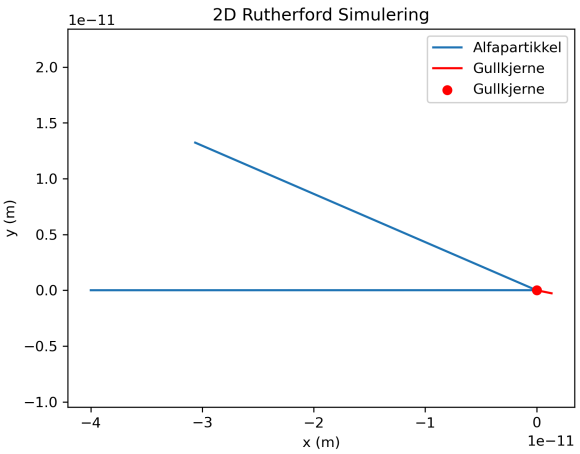
Her starter alfapartikkelen fra punktet $(-4 \cdot 10^{-11}, y)$. Her er y variabelen som bestemmer avstanden fra sentrum av gullatomet prosjektilen skytes. Dette parameteret bestemmer hvor stor avbøyningsvinkel alfapartikkelen får. Radien av gullkjernen er omtrent $7 \cdot 10^{-15}\,\mathrm{m}$. Jeg velger ulike verdier nær dette for å se på avbøyningen.

```
from matplotlib import pyplot as plt
from scipy import constants
import numpy as np

k = 8.99e9

# Mål: gullkjerne
m1, q1 = 79 * constants.m_p + 118 * constants.m_n, 79 *
constants.e
# Prosjektil: alfapartikkel
m2, q2 = 2 * constants.m_p + 2 * constants.m_n, 2 *
constants.e

# Velger gunstig tidsintervall og steg for forsøket
dt = 1e-22
T = 5e-18
MAXN = int(T / dt) + 1

# Definerer DP-tabeller
ts = np.zeros(MAXN)
r1 = np.zeros((MAXN, 2))
r2 = np.zeros((MAXN, 2))
dr1 = np.zeros((MAXN, 2))
dr2 = np.zeros((MAXN, 2))
ddr1 = np.zeros((MAXN, 2))
ddr2 = np.zeros((MAXN, 2))

# Startbetingelser som beskrevet over
r1[0] = [0, 0]
r2[0] = [-4e-11, 5e-15]
dr1[0] = [0, 0]
dr2[0] = [1.5e7, 0]
```

```
for n in range(1, MAXN):
    dr_vec = r2[n-1] - r1[n-1]
    dist = np.linalg.norm(dr_vec)

    # Scalar of force magnitude
    F_mag = k * (q1 * q2) / (dist ** 2)
    force_vec = (dr_vec / dist) * F_mag

    ddr1[n] = -force_vec / m1
    ddr2[n] = force_vec / m2

    dr1[n] = dr1[n-1] + ddr1[n] * dt
    dr2[n] = dr2[n-1] + ddr2[n] * dt

    r1[n] = r1[n-1] + dr1[n] * dt
    r2[n] = r2[n-1] + dr2[n] * dt
```

Dette kan jeg plotte ved

```
fig, ax = plt.subplots()
ax.plot(r1[:, 0], r1[:, 1], color="red",
label="Gullkjerne")
ax.scatter(0, 0, color="red", zorder=5,
label="Gullkjerne")
ax.plot(r2[:, 0], r2[:, 1], label="Alfapartikkel")
ax.set_xlabel("x (m)")
ax.set_ylabel("y (m)")
ax.set_title("2D Rutherford Simulering")
ax.axis("equal")
ax.legend()
plt.show()
```



Figur 8: Simulering av rutherfords forsøk med $y = 5 \cdot 10^{-15}$

2.3 Utregning av vinkel

Jeg kan nå også regne ut avbøyningevinkelen etter å ha simulert forsøket. Jeg har to fartsvektorer v og v_0, som er henholdsvis farten ved starten og slutten av simuleringen. Da kan jeg bruke skalarproduktet til å finne vinkelen:

$$v_0 \cdot v = |v_0| \cdot |v| \cdot \cos\theta$$
$$\cos\theta = \frac{v_0 \cdot v}{|v_0| \cdot |v|}$$
$$\theta = \cos^{-1}\left(\frac{v_0 \cdot v}{|v_0| \cdot |v|}\right)$$

Dette kan jeg regne ut i python med følgende kode:

```
v = dr2[-1]
v0 = dr2[0]
cos_theta = np.dot(v0, v) / (np.linalg.norm(v0) *
np.linalg.norm(v))
theta = np.degrees(np.arccos(np.clip(cos_theta, -1.0,
1.0)))
print(f"Avbøyningsvinkel: {theta:.2f}°")
```

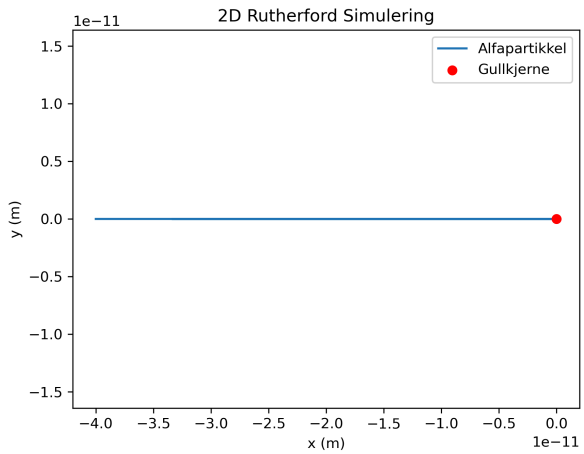
For startverdien av y brukt over får jeg da:

```
Avbøyningsvinkel: 156.63°
```

2.4 Generalisering av funksjonen

Det overraskende resultatet fra Rutherfords eksperiment var at partiklene ikke bare ble avbøyd, men de ble også vinklet så sterkt at de snudde retning og bevegde seg tilbake i retningen de ble skutt. Jeg kan lage en mer generell funksjon basert på koden fra tidligere:

```
def rutherford(y):
  # eksisterende kode for simulering
  # returner de simulerte verdiene
  return r1, r2, theta
```



Figur 9: Simulering av Rutherfords forsøk med $y = 0$

Hvis jeg da bruker støtparameteret $y = 0$, vil jeg se i Figur 9 at partikkelen blir reflektert rett tilbake.

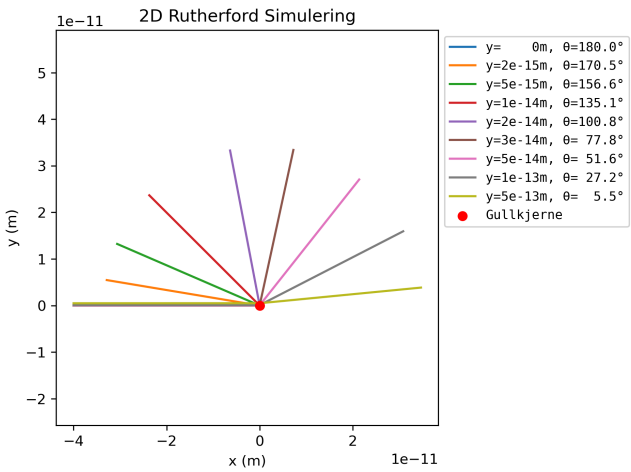
```
r1, r2, theta = rutherford(0)
print(f"Avbøyningsvinkel: {theta}")
```

```
Avbøyningsvinkel: 180.00°
```

2.5 Simulering med flere ulike startverdier

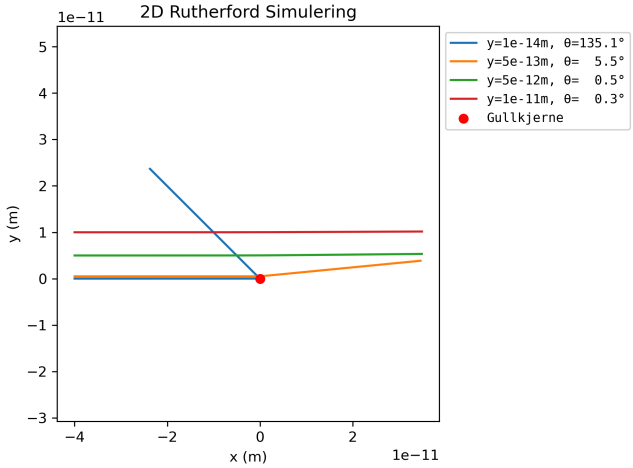
Jeg kan nå bruke den samme koden til å simulere flere ulike verdier for y. Merk at her tegner jeg gullkjernen som et statisk punkt, siden masseforskjellen er så stor at bevegelsen blir minimal.

```
# De ulike verdiene for støtparameteret
ys = [0, 2e-15, 5e-15, 1e-14, 5e-14, 1e-13, 5e-13, 1e-12]

# Opprette en plot der alle linjene skal tegnes
fig, ax = plt.subplots()
ax.set_xlabel("x (m)")
ax.set_ylabel("y (m)")
ax.set_title("2D Rutherford Simulering")
ax.axis("equal")

# Tegne opp en ny linje for hver verdi av y
for y in ys:
    _, r2, theta = rutherford(y)
    ax.plot(
      r2[:, 0],
      r2[:, 1],
      label=f"y={y:>5.0g}m, θ={theta:>5.1f}°"
    )

# Tegne gullkjernen og vise grafen
plt.scatter(0, 0, color="red", zorder=5,
label="Gullkjerne")
ax.legend(
  bbox_to_anchor=(1, 1),
  loc="upper left",
  prop={"family": "monospace", "size": 9}
)
plt.tight_layout()
plt.show()
```



Figur 10: Rutherford-simulering med ulike verdier av y

Vi ser tydelig basert på disse resultatene vist i Figur 10 at større verdier av y fører til lavere avbøyingsvinkler θ. Det vil si at når alfapartikkelen treffer nærmere sentrum av gullkjernen, vil den avbøyes med større vinkel. I Figur 11 ser vi hvordan alfapartiklene ikke blir avbøyd for store nok verdier av y, mens når partiklene nærmer seg atomkjernen, vil de avbøyes mer.



Figur 11: Rutherford-simulering med større verdier av y

3 Konklusjon

I denne oppgaven har jeg simulert kollisjoner mellom ladde partikler i én og to dimensjoner ved hjelp av Euler-Cromers metode. Med disse simuleringene har jeg bekreftet samtlige sentrale prinsipper i fysikken:

- Energibevaring: ved å plotte kinetisk og potensiell energi over tid, har jeg vist at den totale mekaniske energien i systemet forblir konstant.
- Samsvar mellom teori og simulering: Den teoretiske beregnede minsteavstanden ved hjelp av energibevaring og bevegelsesmengde stemte nøyaktig overens med verdiene fra den simulerte modellen.
- Rutherfords atommodell: Ved å utvide den samme modellen til to dimensjoner, har jeg visualisert hvordan alfapartikler avbøyes når de skytes mot en gullkjerne. Simuleringen gjenskaper observasjonene som førte til oppdagelsen av atomkjernen.

Totalt sett viser dette hvor kraftig programmering som et verktøy kan være i fysikken der man må håndtere varierende verdier, og der dette kan settes opp som en differensiallikning.

Kjørbar kode er tilgjengelig på GitHub: https://github.com/LilleAila/projects/tree/main/skole/fysikk1/03-programmeringsoppgave-v26 under `.ipynb`-filer (jupyter notebooks). Kildekoden ligger også vedlagt i en `.zip`-fil.